SUPERSEDED 2026-04-20. First-pass IRC validation snapshot. Current canonical: `IRC_COMPREHENSIVE_2026-04-20.pdf`. Kept for record.

IRC Results Snapshot

Refined GAD → IRC validation, status as of 2026-04-15

What This Note Covers

This note summarizes the *current* IRC validation workflow and the latest scientifically meaningful refined rerun. It is intentionally compact: the goal is to preserve the key numbers, the threshold analysis, and a small amount of interpretation in one place.

Pipeline used here

1. Start from converged GAD trajectories from `noise_survey_300`.
2. These trajectories come from **dataset TS geometries** with Gaussian noise added to the `Transition1x` labeled transition-state coordinates.
3. Select a TS candidate from the saved trajectory (`best_nneg1`).
4. Recompute projected TS quality with HIP: n_{neg}, force norm, and $f_{\max}$.
5. If needed, refine the TS candidate with projected GAD.
6. Run IRC *from that TS* in both downhill directions using Sella's IRC routine.
7. Compare the two endpoints against the `Transition1x` labeled reactant/product using: aligned geometry (element-aware Kabsch/Hungarian RMSD) and bond-topology matching.

Important interpretation point The current labels `intended / half / unintended` mean: *does the HIP IRC from this GAD-found TS recover the dataset-labeled reactant/product pair?* This is a strong benchmark, but not a complete unsupervised analysis of all possible connected minima on HIP's PES.

Run Summary

The refined rerun analyzed here is the completed array `59385481_[0-2]`. It covered 30 converged TS candidates each at noise levels 0, 10, and 50 pm.

Noise	Total	IRC ran	Gate failed	Intended	Half	Topology-half only / Unintended
0 pm	30	15	15	1	3	1 / 10
10 pm	30	7	23	0	2	1 / 4
50 pm	30	3	27	0	0	1 / 2

Headline: the dominant failure mode is now `ts_quality_gate_failed`, not code failure. The looser refined gate did recover one fully intended case at 0 pm and increased IRC attempts there (15 instead of 12), but the attempted cases still become progressively worse with noise: a few mixed / half-intended outcomes at low noise and mostly unintended behavior by 50 pm.

Main Figure

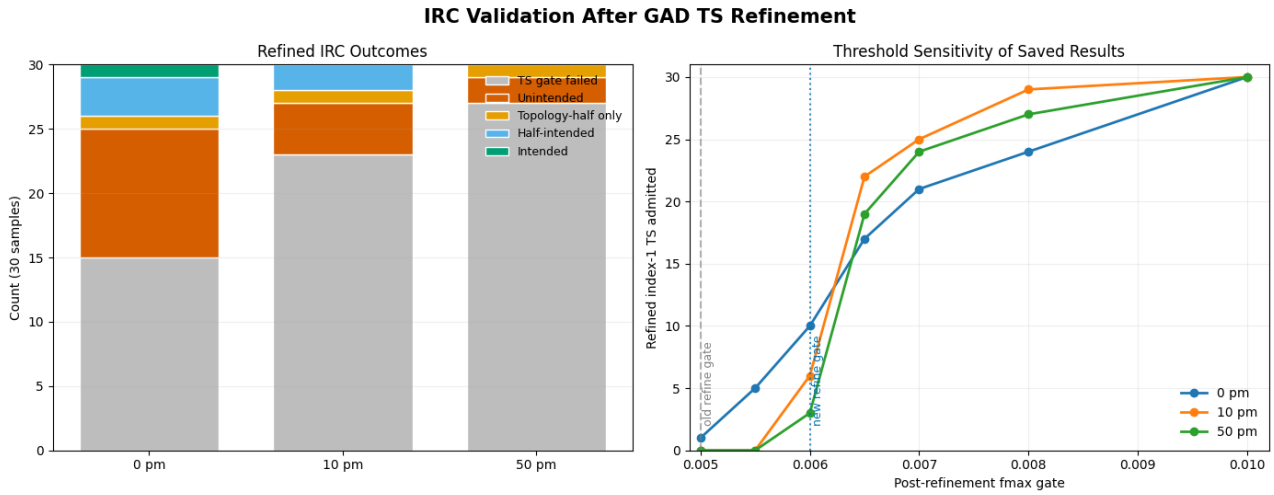

Figure 1: Left: outcome breakdown of the refined rerun. **Right:** number of refined index-1 candidates that would pass under looser post-refinement $f_{\max}$ gates, using the already-saved results.

Interpretation

- Noise hurts in two layers: fewer candidates survive TS screening, and the surviving ones connect less cleanly to the labeled endpoints.
- Even 0 pm is only partially "clean" under the stricter pipeline: one intended case, a few half-intended cases, and many gate failures. This means the old convergence notion ($n_{\text{neg}} = 1$ plus loose force screening) was broad enough that some nominally converged points were not clean TSs by stricter $f_{\max}$-based standards.
- The threshold-sweep panel shows why moving from $f_{\max} < 0.005$ to $f_{\max} < 0.006$ was justified. The current rerun already reflects that change.

Why The Force Gate Was Too Strict

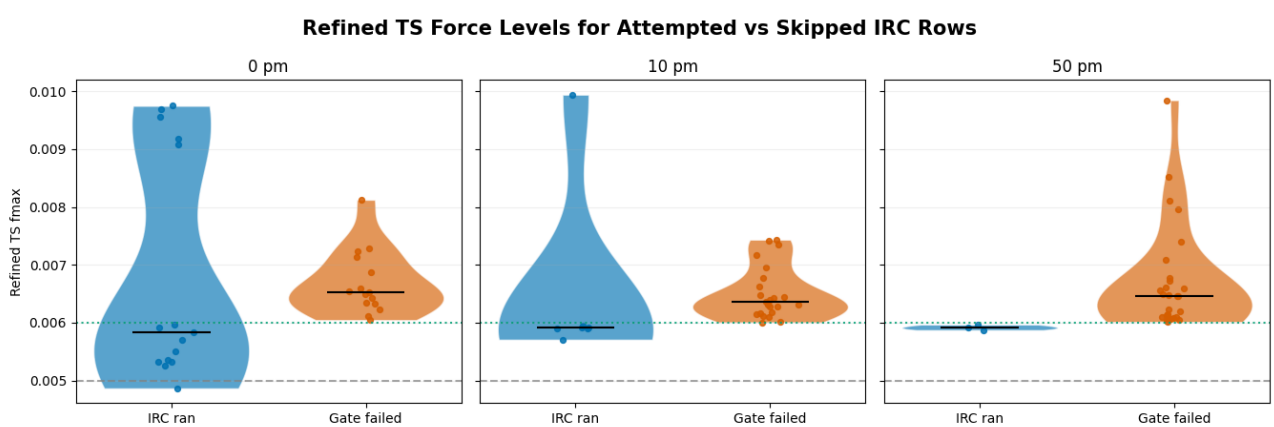

Figure 2: Refined TS $f_{\max}$ for rows that actually ran IRC versus rows that were skipped by the TS-quality gate. Dashed gray: old refined gate 0.005. Dotted green: new refined gate 0.006.

Most important observation: the previous policy had been asymmetric, and the current rerun is

the first one after correcting that.

- Unrefined candidates could go straight to IRC if they satisfied $f_{\max} < 0.01$.
- Refined candidates were only allowed through if they satisfied $f_{\max} < 0.005$.

That old policy meant some *improved* refined TSs were being rejected even though their $f_{\max}$ was smaller than already-accepted unrefined TSs. The current rerun used the corrected post-refinement gate $f_{\max} < 0.006$.

Noise	Max accepted pre-gate $f_{\max}$	Min rejected refined $f_{\max}$	Median rejected refined $f_{\max}$
0 pm	0.009748	0.006202	0.006533
10 pm	0.009941	0.006007	0.006357
50 pm	—	0.006018	0.006456

Conclusion:

- 0.005 was stricter than necessary for a first refined IRC pass.
- The current rerun confirms that $f_{\max} < 0.006$ is a reasonable next operating point: it increases attempted IRC at 0 pm and recovers one intended case without simply reverting to the original 0.01 screen.
- The remaining failed-gate rows are now clustered around 0.0062–0.0065, which is tighter and more interpretable than the old 0.0052–0.0059 bottleneck.

Updated Rerun Settings

The new defaults already patched into the repo are:

- `--refine-steps 600` instead of 300
- `--refine-force-threshold 0.006` instead of 0.005

Why 600 steps? In the completed refined rerun, the refinement stage still occasionally hit the new 600-step cap, especially at 50 pm:

Noise	Rows hitting 600-step refinement cap	Mean refinement steps
0 pm	2	87
10 pm	0	127
50 pm	3	201

So the main gain from the latest rerun came from the looser refined gate rather than dramatically longer refinement trajectories; most cases did not use the full 600-step budget.

What To Inspect First In 3D

The viewer bundles from the completed refined run are already available under: `/lustre07/scratch/memoozd/gadpl`

Recommended cases:

- `0 pm / sample 31`: the one fully intended case in the latest rerun
- `0 pm / samples 0, 33, 45`: representative low-noise half-intended cases
- `10 pm / samples 51, 175`: useful geometry-vs-topology disagreement cases
- `50 pm / sample 175`: topology-half at high noise

- `50 pm / samples 20, 89`: clearly unintended high-noise examples

Each bundle contains:

- dataset reactant reference
- reverse IRC endpoint
- TS input
- forward IRC endpoint
- dataset product reference

So this is an endpoint/context view of the IRC outcome, not a dense movie of every internal IRC integration step.

Bottom Line

- The current refined IRC pipeline is now producing *real* scientific signal.
- The main issue is TS admissibility, not numerical IRC crashes.
- Noise degrades both TS quality and endpoint agreement.
- The latest rerun under $f_{\max} < 0.006$ is better at 0 pm, but high-noise cases remain dominated by gate failures and unintended endpoints.
- The next decision point is not "whether to loosen from 0.005" anymore; that is already done. The next question is whether further loosening or a different TS-selection/refinement strategy is justified.

What Is Running Right Now

Queue snapshot captured at 2026-04-15 17:22 UTC

JobID	Name	State	Time	Node / Reason
59385266_1	run_nr_then_gad.slur	R	1:17:47	ng20601
59385266_2	run_nr_then_gad.slur	R	1:17:47	ng20601
59385266_3	run_nr_then_gad.slur	R	1:17:47	ng30102
59385266_4	run_nr_then_gad.slur	R	1:17:47	ng30102
59385266_5	run_nr_then_gad.slur	R	1:17:47	ng30102
59385266_7	run_nr_then_gad.slur	R	1:17:47	ng20401
59385266_8	run_nr_then_gad.slur	R	1:17:47	ng20401
59385266_9	run_nr_then_gad.slur	R	1:17:47	ng20401
59385266_10	run_nr_then_gad.slur	R	1:14:18	ng30104
59385266_11	run_nr_then_gad.slur	R	1:14:18	ng20401

The latest IRC rerun `59385481_[0-2]` has already completed. The jobs still running now are the `run_nr_then_gad` experiments shown above.